



Today's agenda

↳ miscellaneous problems



AlgoPrep



Q2) Redundant Connection

(bidirectional)

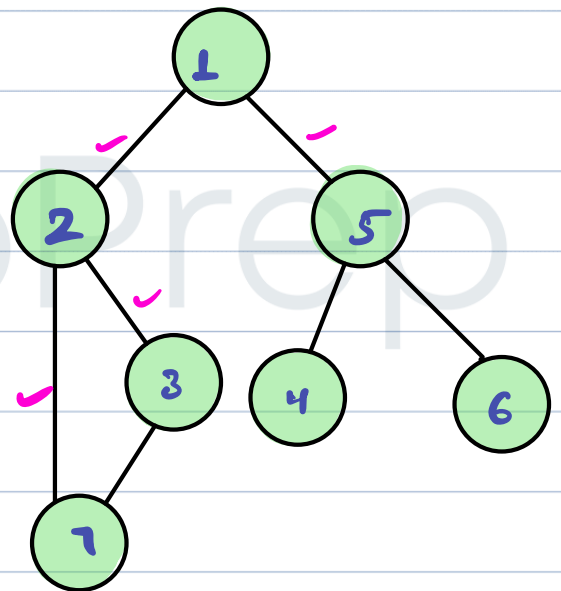
↳ you have been given a tree with 1 extra edge. you have to identify the extra edge.

Note: If multiple answers possible, return the edge occ. last in input.

Idea 1

obs 1: The extra edge is going to create a cycle.

↓
your answer can be any edge of that cycle.



nodes edges

0 1 2

1 1 5

2 2 3

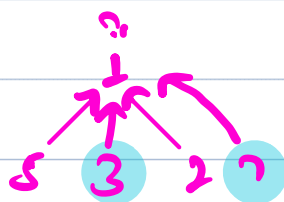
3 2 7

4 3 7

5 5 4

6 5 6

→ ans = 3, 7



4

6



// Pseudo code

```

Public boolean union (int x, int y) {
    int px = find(x);
    int py = find(y);
    if (px == py) { return false; }
    if (rank[px] > rank[py]) {
        Par[py] = px;
    }
    else if (rank[px] < rank[py]) {
        Par[px] = py;
    }
    else {
        Par[px] = py;
        rank[py]++;
    }
    return true;
}

```

union by rank

```

Public int find (int x) {
    if (x == Par[x]) { return x; }
    int temp = find(Par[x]);
    Par[x] = temp; } Path Compression
    return temp;
}

```

^{number of vertices}
 int Par[N+1]; int rank[N+1];

int[] redundantConnection (int[] edges)

```

for (int i=0; i<N; i++) {
    Par[i] = i;
    rank[i] = 1;
}

```

```

for (int i=0; i<edges.length; i++) {
    int u = edges[i][0];
    int v = edges[i][1];

```

```

    boolean flag = union(u, v);

```

```

    if (flag == false) { return edges[i]; }
}

```

```

return new int[2];
}

```

T.C: $O(V+E)$

S.C: $O(V)$



Q) mother vertex

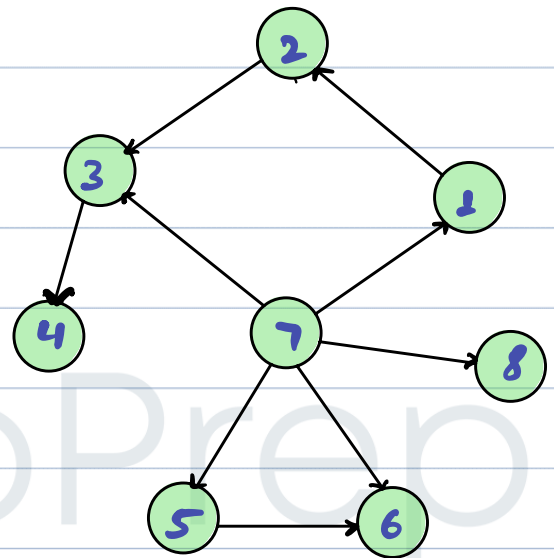
↳ Given a directed graph, find out the mother vertex of the graph.

mother vertex: A mother vertex is a vertex from where you can reach every other vertex.

Idea 1

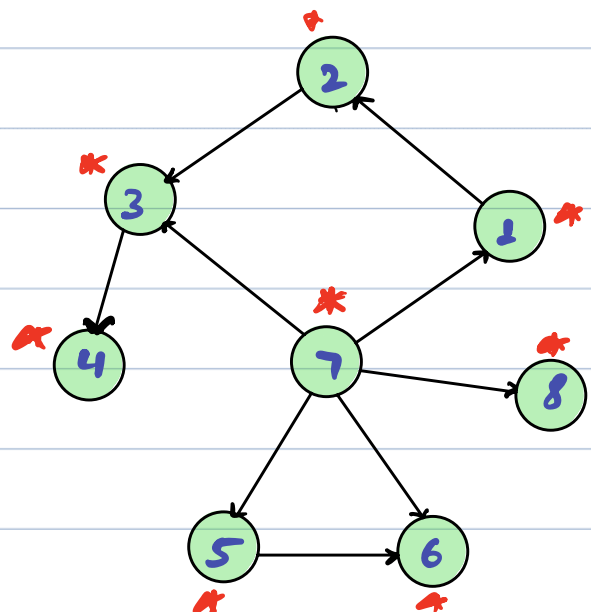
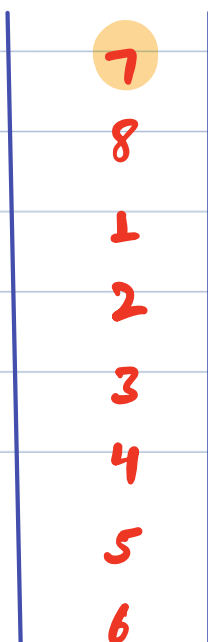
↳ Do any traversal starting from every node one by one.

T.C: $O((V+E)*V)$



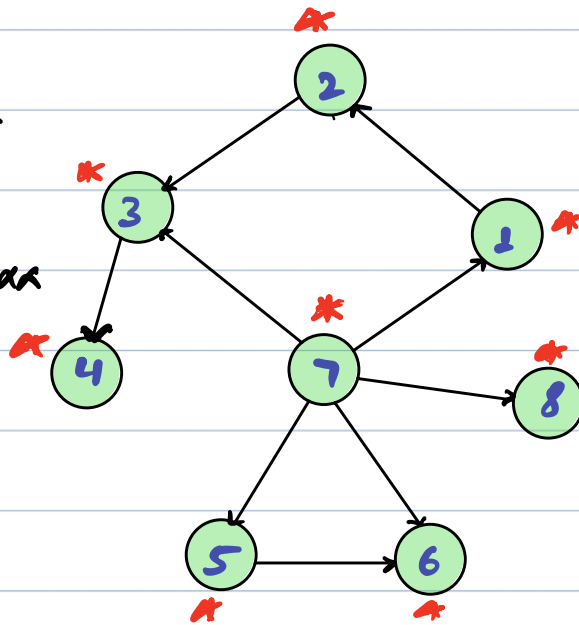
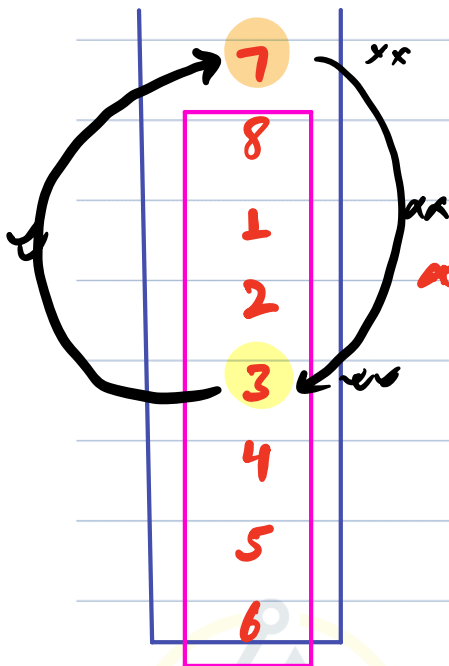
Idea 2

↳ Do dfs and add the node in stack while coming back. The topmost element is your answer.

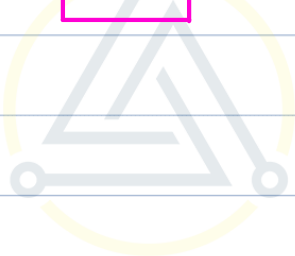




11Proof



ans	tos
x	x
✓	✓
✓	x



AlgoPrep



// Pseudo Code

```
void dfs (List<List<Integer>> graph, boolean[] vis, Stack<int> st, int src)
```

```
    vis[src] = true;
```

```
    List<Integer> nbs = graph.get(src);
```

```
    for (int v: nbs) {
        if (vis[v] == false) {
            dfs(graph, vis, st, v);
        }
    }
```

```
    st.push(src);
```

```
}
```

```
int motherVtn (List<List<Integer>> graph, int v) {
```

```
    // Step 1: Do dfs & store the vtn in stack while coming back.
    boolean vis[v];
```

```
    Stack<Integer> st;
```

```
    for (int i=0; i<v; i++) {
        if (vis[i] == false) {
            dfs(graph, vis, st, i);
        }
    }
```

```
    // Step 2
```

↳ verify if you are able to visit all the nodes from top of stack node.

```
}
```

T.C: $O(V+E)$

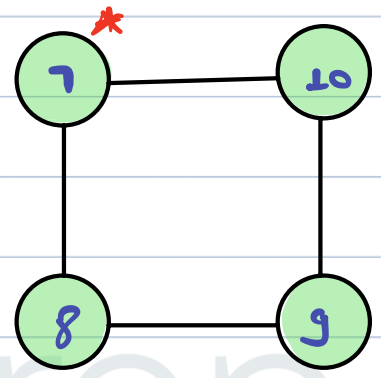
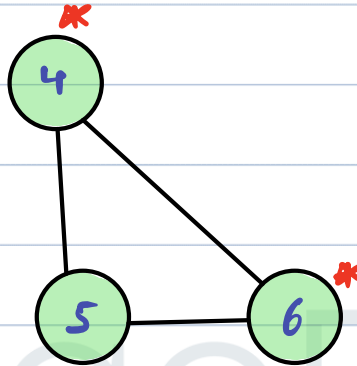
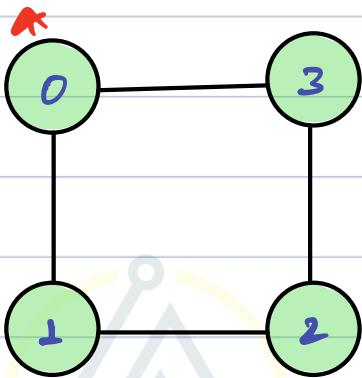
S.C: $O(V)$



Q) minimize malware spread

Given an undirected graph, you have few of nodes infected in the graph (given). find 1 node which if fixed will save the max no. of nodes from infection.

Note: if multiple answers possible, return the smallest node.



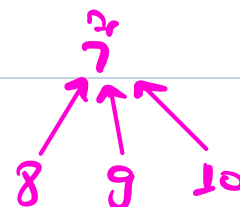
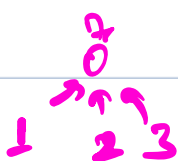
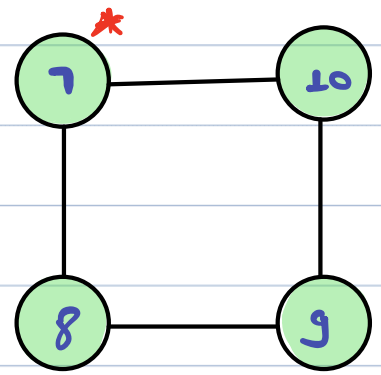
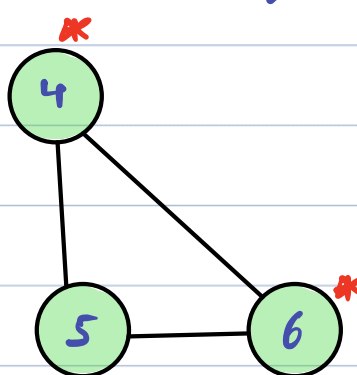
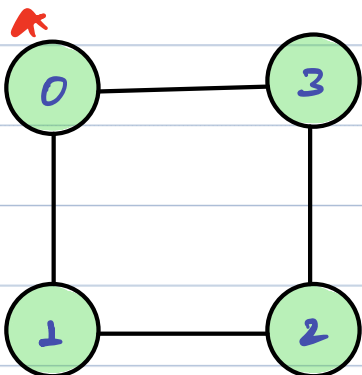
initials: { 0 4 6 7 }

↓ ↓ ↓ ↓

4 0 0 4

Idea 1

DSU on the graph given for set and size.





Size :

0	1	2	3	4	5	6	7	8	9	10
4	0	0	0	3	0	0	4	0	0	0

in/Count :

0	1	2	3	4	5	6	7	8	9	10
1	0	0	0	2	0	0	1	0	0	0

initials: {0 4 6 7}

↳ sort this array

ans idn : 10
ans count = 4

Node	Count of in/Node	Size
0	1	4
4	2	...
6	2	...
7	1	4



1/Pseudo code

```
int Par[N], rank[N];  
int minMalwareSpread (int G[][], int[] initial) {  
    int N = G.length;  
    for (int i = 0; i < N; i++) {  
        Par[i] = i;  
        rank[i] = 1;  
    }  
  
    for (int i = 0; i < G.length; i++) {  
        for (int j = i + 1; j < G.length; j++) {  
            if (G[i][j] == 1) {  
                union(i, j);  
            }  
        }  
    }  
  
    int size[N];  
    for (int i = 0; i < N; i++) {  
        size[find(i)]++;  
    }  
  
    int infCount[N];  
    for (int i = 0; i < initial.length; i++) {  
        int infNode = initial[i];  
        int liNode = find(infNode);  
        infCount[liNode]++;  
    }  
}
```

T.C: $O(N^2 + N \log N)$

S.C: $O(N)$



Arrays. sort (initial);

int ansidn = -1;

int anscount = -∞;

for (int i = 0; i < initial.length; i++) {

int infnode = initial[i];

int linfnod = find(infnod);

if (count[linfnod] == 1 && size[linfnod] > anscount) {

ansidn = infnode;

anscount = size[linfnod];

}

}

return ansidn;

}